

Aufgabe 2

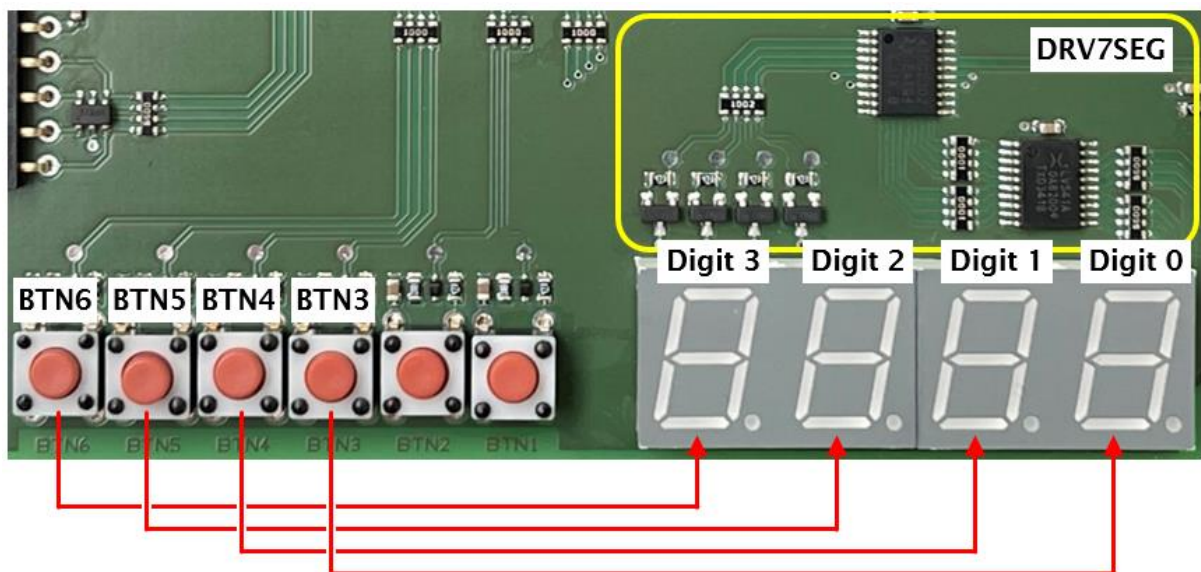
Lernziele: Machen Sie sich mit dem Prinzip der seriellen, synchronen Datenübertragung mittels der SPI-Schnittstelle vertraut. In dieser Aufgabe sollen Sie lernen

- a) wie man Ports des MSP430 als SPI-Schnittstelle konfiguriert,
- b) wie man über die SPI-Schnittstelle auf eine externe SPI-Komponente zugreift,
- c) wie man auf mehrfache Ereignisse in der Funktion `main()` reagiert und
- d) wie man komplexe Abläufe in einer ereignisgesteuerten Applikation in C programmiert.

Die in der 1. Aufgabe implementierten Funktionalitäten (d.h. die Ausgabe eines Blinkmusters sowie das Entprellen von Tasten) sind auch in der 2. Aufgabe zu behalten und ggf. zu erweitern.

Aufgabenstellung

Das Evaluationsboard mit dem Mikrocontroller MSP430FR5729 ist mit einer vierstelligen Siebensegmentanzeige (bestehend aus zwei LTD-4708JR-Modulen) und mit der dazugehörigen Steuerungsschaltung DRV7SEG auf der Basis des Mikrocontrollers MSP430G2302 ausgestattet. Neben der vierstelligen Siebensegmentanzeige mit den Ziffern Digit3, Digit2, Digit1 und Digit0 befinden sich auf dem Evaluationsboard insgesamt sechs Taster (Buttons BTN1 bis BTN6).



Im Rahmen der 2. Aufgabe sind zwei Teilaufgaben zu lösen. 1) Die vierstellige Siebensegmentanzeige ist über die SPI-Schnittstelle des Mikrocontrollers so anzusteuern, dass man mit deren Hilfe beliebige vierstellige Zahlen (von 0000 bis FFFF) anzeigen kann. 2) Mit Hilfe der Taster kann man jede der einzelnen Zifferpositionen separat entweder inkrementieren (+1) oder dekrementieren (-1). Die Auswahl der Funktionen Inkrementieren/Dekrementieren wird über den Taster BTN1 gesteuert, wobei die ausgewählte Funktion gleichzeitig über die LED1 angezeigt wird. Beim Inkrementieren leuchtet die LED1 nicht, und beim Dekrementieren soll sie leuchten. Durch den Klick auf den Taster BTN1 kann man zwischen den beiden Funktionen beliebig hin- und herschalten.

Beim Inkrementieren/Dekrementieren einer Zifferposition ist darauf zu achten, dass ein Überlauf auf die nächsthöhere Zifferposition entstehen kann. Der Überlauf ist auf arithmetisch korrekte Art zu behandeln und ggf. auf die darauffolgenden Zifferpositionen weiter zu leiten. In der unteren Tabelle sind

einige Beispiele im dezimalen Zahlensystem (also zur Basis 10) dargestellt, wie sich das System beim Inkrementieren/Dekrementieren zu verhalten hat.

Inkrementieren					
Anzeige VOR	BTN				Anzeige DANACH
	6	5	4	3	
0000				X	0001
0001				X	0002
0009				X	0010
1299				X	1300
0030			X		0040
1299			X		1309
9999				X	0000
1234		X			1334
1934		X			2034
9999		X			0099
8765	X				9765
9765	X				0765

Dekrementieren					
Anzeige VOR	BTN				Anzeige DANACH
	6	5	4	3	
0000				X	9999
0001				X	0000
0009				X	0008
1300				X	1299
0040			X		0030
1309			X		1299
0099		X			9999
1334		X			1234
2034		X			1934
2004			X		1994
0099		X			9999
9765	X				8765

Die Basis des Zahlensystems wird mit einer #define-Konstante festgelegt und kann mit den Werten zwischen 2 bis 16 vor dem Kompilieren belegt werden. Die Applikation muss in der Lage sein, vierstellige Zahlen zu einer beliebigen Basis zwischen 2 und 16 korrekt inkrementieren und dekrementieren zu können.

Nach der Reset/Power-On-Phase ist die Funktion Inkrementieren aktiv, die vierstellige Zahl wird mit 0000 initialisiert und die vierstellige Siebensegmentanzeige zeigt den Wert 0000 an.

Hinweise zur Implementierung: 1) Der Ablauf kann in zwei oder drei sog. Handler aufgeteilt werden, die mittels Events und Daten miteinander kommunizieren und kooperieren. Beispielsweise könnte sich ein zustandsloser Handler um die Auswertung von Button-Events kümmern, ein zweiter Handler um die Aktualisierung der einzelnen Zifferpositionen. Ein dritter, zustandsbehafteter Handler kümmert sich um die Ausgabe der vierstelligen Zahl mittels der SPI-Schnittstelle und muss dann mit Hilfe einer Zustandsmaschine implementiert werden. Die Funktionalitäten aus dem ersten und zweiten Handler können in einem Handler zusammengefasst werden. Sind alle Handler „sauber“ implementiert, so kann man ihre Reihenfolge in der Funktion main() beliebig vertauschen. 2) Schleifen (egal wie implementiert, ob mit Hilfe von while, for oder goto) sind in der implementierten Aufgabe nicht zulässig. Von dieser Einschränkung sind Initialisierungssequenzen ausgenommen. Bei der Abnahme der Aufgabe wird darauf geachtet. Werden Schleifen gefunden, so gilt die Aufgabe als nicht bestanden. Bei der Benutzung von Funktionen aus C-Bibliotheken muss man vorsichtig sein. Auch solche Funktionen beinhalten häufig Schleifen. Komplexe arithmetische Operationen wie Multiplikation, Division oder Modulo werden mit Hilfe von Schleifen realisiert und sind bei der Lösung der Aufgabe nicht zulässig. 3) Auch der „Spaghetticode“ ist nicht zulässig. Manche versuchen, eine Schleife durch mehrfaches Kopieren des Schleifenrumpfes hintereinander zu ersetzen.